Algorithms 1-3 present the detailed pseudocode outlining our proposed SCALA pipeline for Intelligent Fault Resolution.

Algorithm 1 GenerateKB

Input: Fault log corpus $\mathcal{F}$
Output: Augmented fault log knowledge base $\mathcal{K}$
/* Encoder Stage */
Fine-tune MiniLM encoder E using corpus $\mathcal{F}$
Extract fault embeddings $\mathcal{E} = \{e_i = E(f_i)\}_{i=1}^{N}$ where $f_i \in \mathcal{F}$ and $e_i \in \mathbb{R}^{384}$
/* MAD-GAN Stage */
Initialize K generators $\{G_1, G_2, ..., G_K\}$ and one discriminator D
for number of training epochs **do**
    Sample real embeddings $e_i \in \mathcal{E}$
    for $j = 1$ to K **do**
        Generate fake embedding $\tilde{e}_j = G_j(z),\ z \sim \mathcal{N}(0, I)$
    end for
    Train D to (i) distinguish real vs fake, and (ii) identify generator G_j for $\tilde{e}_j$
    Train each G_j to fool D into classifying $\tilde{e}_j$ as real
end for
Trained Generator G produces synthetic embeddings $\{\tilde{e}_j\}$
/* Decoder Stage */
Fine-tune GPT-2 decoder Dec using corpus $\mathcal{F}$
Train a neural projector $P : \mathbb{R}^{384} \to \mathbb{R}^{768}$ to map encoder space to decoder space
Freeze Dec weights and train projector P using pairs (e_i, f_i)
Generate synthetic fault logs $\tilde{f}_j = Dec(P(\tilde{e}_j))$
$\mathcal{K} \leftarrow \mathcal{K} \cup \{(\tilde{f}_j)\}$
return $\mathcal{K}$

Algorithm 2 Intelligent Fault Resolution (Generator)

Input: New fault description f_{new} and retrieval size k
Output: Context-aware resolution $r_{\texttt{new}}$
$\mathcal{K} \leftarrow$ **GenerateKB** Algorithm()
$\mathcal{C} \leftarrow$ **Perform GMM Clustering**($\mathcal{K}$)
`new_fault` $\leftarrow f_{\texttt{new}}$
`new_embed` $\leftarrow$ **Fault Embedding**(`new_fault`)
$c^* \leftarrow$ **Cluster Prediction**(`new_embed`, $\mathcal{C}$)
$(closer\_faults,\ corres\_resolutions) \leftarrow$ **GCF***Algorithm*($new\_embed,\ \mathcal{K},\ c^*,\ k$)
$prompt \leftarrow$ **ConstructPrompt**($closer\_faults,\ corres\_resolutions,\ new\_fault$)
$r_{new} \leftarrow$ **GPT**($prompt$)
return r_{new}

Algorithm 3 Get Closer Faults (**GCF**) Retriever

```
function GET_CLOSER_FAULTS(new_embed, 𝒦, c*, k)
    ℱ_{c*} ← filter faults from 𝒦 belonging to cluster c*
    existing_faults ← fault descriptions from ℱ_{c*}
    existing_embed ← Fault Embedding(existing_faults)
    closer ← CosineSimilarity(new_embed, existing_embed)
    top_indices ← select indices of k closest faults
                  with highest similarity scores
    closer_faults ← [existing_faults[j]
                     for j in top_indices]
    corres_resolutions ← [resolution[j]
                          for j in top_indices]
    return (closer_faults, corres_resolutions)
end function
```